

Chapter 4: Value Iteration and Policy Iteration

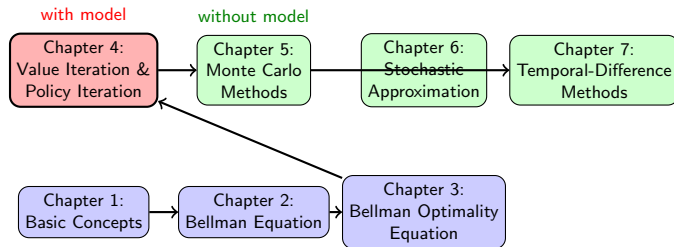
Reinforcement Learning

Adapted from Shiyu Zhao's textbook

Outline

- 1 Introduction
- 2 Value Iteration
- 3 Policy Iteration
- 4 Truncated Policy Iteration
- 5 Summary
- 6 Q&A

Where We Are in This Course



This chapter: First algorithms that can find optimal policies!

- Value Iteration
- Policy Iteration
- Truncated Policy Iteration

Overview of This Chapter

Three closely related algorithms:

① Value Iteration

- Directly suggested by the contraction mapping theorem
- Solves the Bellman optimality equation

② Policy Iteration

- Idea widely used in reinforcement learning
- Alternates between policy evaluation and policy improvement

③ Truncated Policy Iteration

- Unified algorithm
- Value iteration and policy iteration are special cases

Important: These are **dynamic programming** algorithms that require the system model.

Value Iteration: The Core Idea

Recall from Chapter 3: The Bellman optimality equation is

$$v^* = \max_{\pi} (r_{\pi} + \gamma P_{\pi} v^*)$$

Contraction Mapping Theorem suggests the following iterative algorithm:

$$v_{k+1} = \max_{\pi \in \Pi} (r_{\pi} + \gamma P_{\pi} v_k), \quad k = 0, 1, 2, \dots$$

Theorem 3.3 guarantees:

- $v_k \rightarrow v^*$ as $k \rightarrow \infty$ (optimal state value)
- $\pi_k \rightarrow \pi^*$ as $k \rightarrow \infty$ (optimal policy)

Value Iteration: Two Steps per Iteration

Each iteration consists of **two steps**:

Step 1: Policy Update

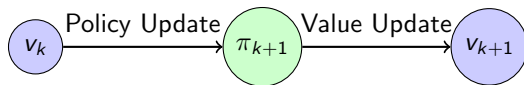
$$\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k)$$

Find the policy that maximizes the expected return given current value estimate v_k .

Step 2: Value Update

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

Compute the new value using the updated policy.



Elementwise Form: Policy Update Step

For each state $s \in \mathcal{S}$, the policy update is:

$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k(s') \right)}_{q_k(s, a)}$$

The optimal policy that solves this is **greedy**:

$$\pi_{k+1}(a|s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases}$$

where $a_k^*(s) = \arg \max_a q_k(s, a)$.

Key insight: The new policy simply selects the action with the greatest q -value.

Note: If $\arg \max_a q_k(s, a)$ has multiple solutions, select any of them.

Elementwise Form: Value Update Step

For each state $s \in \mathcal{S}$, the value update is:

$$v_{k+1}(s) = \sum_a \pi_{k+1}(a|s) \underbrace{\left(\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k(s') \right)}_{q_k(s, a)}$$

Since π_{k+1} is greedy, substituting gives:

$$v_{k+1}(s) = \max_a q_k(s, a)$$

Summary of the process:

$$v_k(s) \longrightarrow q_k(s, a) \longrightarrow \pi_{k+1}(s) \longrightarrow v_{k+1}(s) = \max_a q_k(s, a)$$

Value Iteration Algorithm

Algorithm: Value Iteration

Input: Probability models $p(r|s, a)$ and $p(s'|s, a)$, initial guess v_0

Goal: Find optimal state value v^* and optimal policy π^*

While $\|v_k - v_{k-1}\| > \epsilon$ (threshold), do:

- **For** every state $s \in \mathcal{S}$:
 - **For** every action $a \in \mathcal{A}(s)$:

$$q_k(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k(s')$$

- $a_k^*(s) = \arg \max_a q_k(s, a)$
- $\pi_{k+1}(a|s) = 1$ if $a = a_k^*$, else $\pi_{k+1}(a|s) = 0$
- $v_{k+1}(s) = \max_a q_k(s, a)$

Important Note: v_k is NOT a State Value!

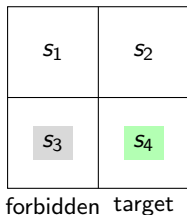
Question: Is v_k a state value?

Answer: No!

- Although v_k eventually converges to the optimal state value v^*
- It is **not** guaranteed to satisfy the Bellman equation of any policy
- In general:
 - $v_k \neq r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$
 - $v_k \neq r_{\pi_k} + \gamma P_{\pi_k} v_k$
- v_k is merely an **intermediate value** generated by the algorithm

Similarly: Since v_k is not a state value, q_k is not an action value.

Example: 2×2 Grid World



Setup:

- Target area: s_4
- Forbidden area: s_3 (gray)
- Rewards:
 - $r_{\text{boundary}} = r_{\text{forbidden}} = -1$
 - $r_{\text{target}} = 1$
- Discount rate: $\gamma = 0.9$
- Actions: $\{a_1, a_2, a_3, a_4, a_5\}$ (up, down, right, left, stay)

Example: Q-Table Expression

The q -value for each state-action pair:

$q(s, a)$	a_1	a_2	a_3	a_4	a_5
s_1	$-1 + \gamma v(s_1)$	$-1 + \gamma v(s_2)$	$0 + \gamma v(s_3)$	$-1 + \gamma v(s_1)$	$0 + \gamma v(s_1)$
s_2	$-1 + \gamma v(s_2)$	$-1 + \gamma v(s_2)$	$1 + \gamma v(s_4)$	$0 + \gamma v(s_1)$	$-1 + \gamma v(s_2)$
s_3	$0 + \gamma v(s_1)$	$1 + \gamma v(s_4)$	$-1 + \gamma v(s_3)$	$-1 + \gamma v(s_3)$	$0 + \gamma v(s_3)$
s_4	$-1 + \gamma v(s_2)$	$-1 + \gamma v(s_4)$	$-1 + \gamma v(s_4)$	$0 + \gamma v(s_3)$	$1 + \gamma v(s_4)$

Example: Iteration $k = 0$

Initial values: $v_0(s_1) = v_0(s_2) = v_0(s_3) = v_0(s_4) = 0$

Q-value calculation:

$q_0(s, a)$	a_1	a_2	a_3	a_4	a_5
s_1	-1	-1	0	-1	0
s_2	-1	-1	1	0	-1
s_3	0	1	-1	-1	0
s_4	-1	-1	-1	0	1

Policy update: $\pi_1(a_5|s_1) = 1$, $\pi_1(a_3|s_2) = 1$, $\pi_1(a_2|s_3) = 1$, $\pi_1(a_5|s_4) = 1$

Value update: $v_1(s_1) = 0$, $v_1(s_2) = 1$, $v_1(s_3) = 1$, $v_1(s_4) = 1$

Note: Policy π_1 is NOT optimal (stays at s_1)!

Example: Iteration $k = 1$

Current values: $v_1(s_1) = 0$, $v_1(s_2) = 1$, $v_1(s_3) = 1$, $v_1(s_4) = 1$

Q-value calculation:

$q_1(s, a)$	a_1	a_2	a_3	a_4	a_5
s_1	$-1 + 0.9 \cdot 0$	$-1 + 0.9 \cdot 1$	$0 + 0.9 \cdot 1$	$-1 + 0.9 \cdot 0$	$0 + 0.9 \cdot 0$
s_2	$-1 + 0.9 \cdot 1$	$-1 + 0.9 \cdot 1$	$1 + 0.9 \cdot 1$	$0 + 0.9 \cdot 0$	$-1 + 0.9 \cdot 1$
s_3	$0 + 0.9 \cdot 0$	$1 + 0.9 \cdot 1$	$-1 + 0.9 \cdot 1$	$-1 + 0.9 \cdot 1$	$0 + 0.9 \cdot 1$
s_4	$-1 + 0.9 \cdot 1$	$-1 + 0.9 \cdot 1$	$-1 + 0.9 \cdot 1$	$0 + 0.9 \cdot 1$	$1 + 0.9 \cdot 1$

Policy update: $\pi_2(a_3|s_1) = 1$, $\pi_2(a_3|s_2) = 1$, $\pi_2(a_2|s_3) = 1$, $\pi_2(a_5|s_4) = 1$

Value update: $v_2(s_1) = 0.9$, $v_2(s_2) = 1.9$, $v_2(s_3) = 1.9$, $v_2(s_4) = 1.9$

Policy π_2 is already optimal! (Only 2 iterations needed)

Policy Iteration: Overview

Policy iteration is another important algorithm for finding optimal policies.

Key difference from value iteration:

- Not for directly solving the Bellman optimality equation
- But has an intimate relationship with value iteration
- Idea is widely used in reinforcement learning algorithms

Each iteration has two steps:

- 1 **Policy Evaluation (PE):** Calculate the state value v_{π_k} for current policy π_k
- 2 **Policy Improvement (PI):** Find a better policy π_{k+1}

Policy Iteration: Two Steps

Step 1: Policy Evaluation

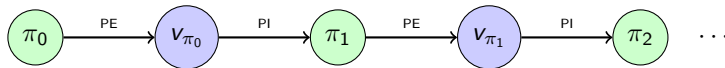
Given policy π_k , solve for its state value by solving the Bellman equation:

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

Step 2: Policy Improvement

Given v_{π_k} , find a new (better) policy:

$$\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k})$$



Three Key Questions

Three natural questions:

- ① **Policy Evaluation:** How to solve for v_{π_k} ?
- ② **Policy Improvement:** Why is π_{k+1} better than π_k ?
- ③ **Convergence:** Why does this algorithm converge to an optimal policy?

Let's answer these questions one by one!

Q1: How to Calculate v_{π_k} ?

Method 1: Closed-form solution

$$v_{\pi_k} = (I - \gamma P_{\pi_k})^{-1} r_{\pi_k}$$

- Useful for theoretical analysis
- Inefficient to implement (requires matrix inverse)

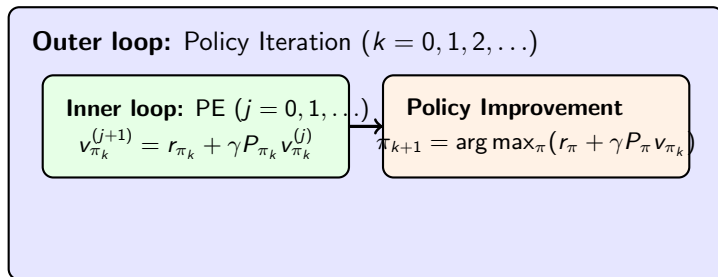
Method 2: Iterative algorithm (preferred in practice)

$$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \dots$$

- Start from any initial guess $v_{\pi_k}^{(0)}$
- Guaranteed: $v_{\pi_k}^{(j)} \rightarrow v_{\pi_k}$ as $j \rightarrow \infty$
- Easy to implement!

Nested Iterative Structure

Key observation: Policy iteration has an **iterative algorithm embedded** inside!



In theory: Inner loop requires $j \rightarrow \infty$ to get exact v_{π_k}

In practice: Stop when $\|v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)}\| < \epsilon$ or j exceeds a limit

Good news: Imprecise v_{π_k} still works! (Explained later)

Q2: Why is π_{k+1} Better Than π_k ?

Lemma 4.1 (Policy Improvement)

If $\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k})$, then

$$v_{\pi_{k+1}} \geq v_{\pi_k}$$

where $v_{\pi_{k+1}} \geq v_{\pi_k}$ means $v_{\pi_{k+1}}(s) \geq v_{\pi_k}(s)$ for all s .

Intuition: The new policy π_{k+1} is chosen to maximize the expected return given the current value estimate. This guarantees improvement!

Proof of Policy Improvement Lemma

Proof:

Since $v_{\pi_{k+1}}$ and v_{π_k} satisfy their respective Bellman equations:

$$v_{\pi_{k+1}} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_{k+1}}$$

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

Since $\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k})$:

$$r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_k} \geq r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

Therefore:

$$\begin{aligned} v_{\pi_k} - v_{\pi_{k+1}} &= (r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}) - (r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_{k+1}}) \\ &\leq (r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_k}) - (r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_{k+1}}) \\ &= \gamma P_{\pi_{k+1}} (v_{\pi_k} - v_{\pi_{k+1}}) \end{aligned}$$

Proof of Policy Improvement Lemma (cont.)

Continuing from $v_{\pi_k} - v_{\pi_{k+1}} \leq \gamma P_{\pi_{k+1}}(v_{\pi_k} - v_{\pi_{k+1}})$:

By recursion:

$$\begin{aligned} v_{\pi_k} - v_{\pi_{k+1}} &\leq \gamma^2 P_{\pi_{k+1}}^2 (v_{\pi_k} - v_{\pi_{k+1}}) \\ &\leq \dots \\ &\leq \gamma^n P_{\pi_{k+1}}^n (v_{\pi_k} - v_{\pi_{k+1}}) \\ &\leq \lim_{n \rightarrow \infty} \gamma^n P_{\pi_{k+1}}^n (v_{\pi_k} - v_{\pi_{k+1}}) = 0 \end{aligned}$$

The limit is zero because:

- $\gamma^n \rightarrow 0$ as $n \rightarrow \infty$ (since $\gamma < 1$)
- $P_{\pi_{k+1}}^n$ is a nonnegative stochastic matrix

Conclusion: $v_{\pi_k} - v_{\pi_{k+1}} \leq 0$, i.e., $v_{\pi_{k+1}} \geq v_{\pi_k}$



Q3: Why Does Policy Iteration Converge?

Policy iteration generates two sequences:

- Policies: $\{\pi_0, \pi_1, \dots, \pi_k, \dots\}$
- State values: $\{v_{\pi_0}, v_{\pi_1}, \dots, v_{\pi_k}, \dots\}$

Key properties:

- $v_{\pi_k} \leq v^*$ for all k (bounded above by optimal)
- By Lemma 4.1: $v_{\pi_0} \leq v_{\pi_1} \leq v_{\pi_2} \leq \dots \leq v_{\pi_k} \leq \dots \leq v^*$

By Monotone Convergence Theorem:

Since $\{v_{\pi_k}\}$ is nondecreasing and bounded above, it converges!

Theorem 4.1 (Convergence of Policy Iteration)

The state value sequence $\{v_{\pi_k}\}_{k=0}^{\infty}$ converges to v^* .

The policy sequence $\{\pi_k\}_{k=0}^{\infty}$ converges to an optimal policy.

Proof Idea: Policy Iteration Converges Faster Than Value Iteration

Key insight: Policy iteration converges at least as fast as value iteration.

Setup: Let both algorithms start from same initial condition: $v_0 = v_{\pi_0}$

We can show by induction:

$$v_k \leq v_{\pi_k} \leq v^* \quad \text{for all } k \geq 0$$

where $\{v_k\}$ is generated by value iteration.

Since:

- Value iteration: $v_k \rightarrow v^*$ (guaranteed by contraction mapping theorem)
- $v_k \leq v_{\pi_k} \leq v^*$

Therefore: $v_{\pi_k} \rightarrow v^*$ as well!



Policy Iteration: Elementwise Form

Policy Evaluation (elementwise):

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}^{(j)}(s') \right]$$

for $j = 0, 1, 2, \dots$ and all $s \in \mathcal{S}$.

Policy Improvement (elementwise):

$$\pi_{k+1}(a|s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases}$$

where $a_k^*(s) = \arg \max_a q_{\pi_k}(s, a)$ and

$$q_{\pi_k}(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}(s')$$

Policy Iteration Algorithm

Algorithm: Policy Iteration

Input: System model $p(r|s, a)$, $p(s'|s, a)$; initial policy π_0

While v_{π_k} has not converged, do:

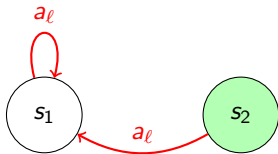
Policy Evaluation:

- Initialize $v_{\pi_k}^{(0)}$ arbitrarily
- **While** $v_{\pi_k}^{(j)}$ not converged:
 - For all s : $v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}^{(j)}(s') \right]$

Policy Improvement:

- For all s, a : $q_{\pi_k}(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}(s')$
- For all s : $\pi_{k+1}(a|s) = 1$ if $a = \arg \max_a q_{\pi_k}(s, a)$, else 0

Example: Simple Two-State MDP



(a) Initial π_0

Setup:

- States: $\{s_1, s_2\}$, target: s_2
- Actions: $\{a_\ell, a_0, a_r\}$ (left, stay, right)
- Rewards: $r_{\text{boundary}} = -1$, $r_{\text{target}} = 1$
- Discount: $\gamma = 0.9$

Initial policy π_0 : Move left at both states (bad!)

Example: Policy Evaluation Step

Step 1: Policy Evaluation for π_0

Solve the Bellman equation:

$$v_{\pi_0}(s_1) = -1 + \gamma v_{\pi_0}(s_1)$$

$$v_{\pi_0}(s_2) = 0 + \gamma v_{\pi_0}(s_1)$$

Solution:

$$v_{\pi_0}(s_1) = \frac{-1}{1 - 0.9} = -10, \quad v_{\pi_0}(s_2) = 0.9 \times (-10) = -9$$

Or iteratively: Starting with $v^{(0)} = 0$:

$$v_{\pi_0}^{(1)}(s_1) = -1, \quad v_{\pi_0}^{(1)}(s_2) = 0$$

$$v_{\pi_0}^{(2)}(s_1) = -1.9, \quad v_{\pi_0}^{(2)}(s_2) = -0.9$$

$$v_{\pi_0}^{(3)}(s_1) = -2.71, \quad v_{\pi_0}^{(3)}(s_2) = -1.71$$

$$\vdots \rightarrow (-10, -9)$$

Example: Policy Improvement Step

Step 2: Policy Improvement

Compute $q_{\pi_0}(s, a)$ using the q-table:

$q_{\pi_k}(s, a)$	a_ℓ	a_0	a_r
s_1	$-1 + \gamma v_{\pi_k}(s_1)$	$0 + \gamma v_{\pi_k}(s_1)$	$1 + \gamma v_{\pi_k}(s_2)$
s_2	$0 + \gamma v_{\pi_k}(s_1)$	$1 + \gamma v_{\pi_k}(s_2)$	$-1 + \gamma v_{\pi_k}(s_2)$

Substituting $v_{\pi_0}(s_1) = -10$, $v_{\pi_0}(s_2) = -9$:

$q_{\pi_0}(s, a)$	a_ℓ	a_0	a_r
s_1	-10	-9	-7.1
s_2	-9	-7.1	-9.1

New policy: $\pi_1(a_r|s_1) = 1$, $\pi_1(a_0|s_2) = 1$ (Optimal!)

Comparing Value Iteration and Policy Iteration

Policy Iteration:

- PE: Given π_k , solve $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$
- PI: Given v_{π_k} , solve $\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k})$

Value Iteration:

- PU: Given v_k , solve $\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k)$
- VU: Given π_{k+1} , compute $v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$

Policy iteration: $\pi_0 \xrightarrow{\text{PE}} v_{\pi_0} \xrightarrow{\text{PI}} \pi_1 \xrightarrow{\text{PE}} v_{\pi_1} \xrightarrow{\text{PI}} \pi_2 \dots$

Value iteration: $v_0 \xrightarrow{\text{PU}} \pi'_1 \xrightarrow{\text{VU}} v_1 \xrightarrow{\text{PU}} \pi'_2 \xrightarrow{\text{VU}} v_2 \dots$

The Key Difference: Number of Inner Iterations

Focus on the value step (assuming $v_0 = v_{\pi_0}$):

Policy iteration solves $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$ iteratively:

$$v_{\pi_1}^{(0)} = v_0$$

Value iteration: $v_1 \leftarrow$

$$v_{\pi_1}^{(1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)}$$

$$v_{\pi_1}^{(2)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)}$$

$\vdots$

$$v_{\pi_1}^{(j)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)}$$

Truncated: $\bar{v}_1 \leftarrow$

Policy iteration: $v_{\pi_1} \leftarrow$

Truncated Policy Iteration: The Unified View

Truncated Policy Iteration:

- Run j_{truncate} iterations in the policy evaluation step
- Not infinite (like policy iteration)
- Not just one (like value iteration)

Key Insight

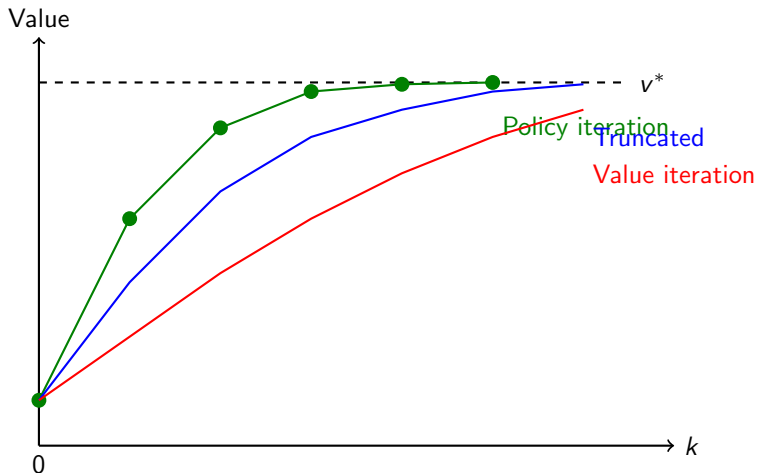
Value iteration and policy iteration are **two extreme cases** of truncated policy iteration:

- $j_{\text{truncate}} = 1 \Rightarrow$ Value Iteration
- $j_{\text{truncate}} = \infty \Rightarrow$ Policy Iteration

Trade-off:

- More inner iterations $\Rightarrow$ faster outer convergence
- Fewer inner iterations $\Rightarrow$ less computation per outer iteration

Convergence Comparison



Observation: Policy iteration converges fastest, value iteration slowest, truncated in between.

Truncated Policy Iteration Algorithm

Algorithm: Truncated Policy Iteration

Input: System model, initial policy π_0 , truncation parameter j_{truncate}

While v_k not converged, do:

Policy Evaluation (truncated):

- Set $v_k^{(0)} = v_{k-1}$
- **For** $j = 0, 1, \dots, j_{\text{truncate}} - 1$:
 - For all s : $v_k^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k^{(j)}(s') \right]$
- Set $v_k = v_k^{(j_{\text{truncate}})}$

Policy Improvement:

- For all s, a : $q_k(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k(s')$
- $\pi_{k+1}(a|s) = 1$ if $a = \arg \max_a q_k(s, a)$, else 0

Value Improvement Property

Proposition 4.1 (Value Improvement)

In the policy evaluation step: $v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}$

If the initial guess is $v_{\pi_k}^{(0)} = v_{\pi_{k-1}}$, then:

$$v_{\pi_k}^{(j+1)} \geq v_{\pi_k}^{(j)} \quad \text{for } j = 0, 1, 2, \dots$$

Proof sketch:

- 1 Show: $v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)} = \gamma^j P_{\pi_k}^j (v_{\pi_k}^{(1)} - v_{\pi_k}^{(0)})$
- 2 Show: $v_{\pi_k}^{(1)} \geq v_{\pi_k}^{(0)} = v_{\pi_{k-1}}$ (due to policy improvement)
- 3 Since $P_{\pi_k}^j \geq 0$, we have $v_{\pi_k}^{(j+1)} \geq v_{\pi_k}^{(j)}$

Implication: Values are monotonically improving, even with truncation!

Advantages of Truncated Policy Iteration

Compared to Policy Iteration:

- Only requires finite iterations in policy evaluation
- More computationally efficient per outer iteration
- Imprecise v_k still leads to convergence

Compared to Value Iteration:

- Faster convergence rate
- A few extra inner iterations significantly speed up overall convergence

Practical guideline:

- Run a few iterations (e.g., 5-20) in policy evaluation
- Too few: slow like value iteration
- Too many: diminishing returns

Summary: Three Algorithms

	Value Iteration	Truncated PI	Policy Iteration
Inner iterations	1	j_{truncate}	∞
v_k is state value?	No	No	Yes
Convergence speed	Slowest	Middle	Fastest
Computation/iter	Lowest	Middle	Highest

Common property: Every iteration has two steps

- Value update step
- Policy update step

This idea of **interaction between value and policy** is called **Generalized Policy Iteration (GPI)** and is fundamental to RL!

Key Takeaways

① Value Iteration

- Directly solves Bellman optimality equation
- Guaranteed by contraction mapping theorem
- v_k are NOT state values

② Policy Iteration

- Alternates between policy evaluation and improvement
- v_{π_k} ARE state values
- Converges faster but more computation per iteration

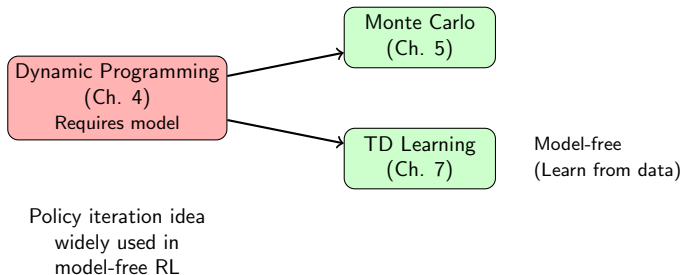
③ Truncated Policy Iteration

- Unified framework
- Balances speed and computation
- Most practical choice

All three require the system model $\Rightarrow$ Dynamic Programming algorithms

Next: Model-free algorithms (Monte Carlo, TD, etc.)

Looking Ahead



Key insight: The model-free algorithms in subsequent chapters can be viewed as extensions of the algorithms in this chapter!

Frequently Asked Questions (1/3)

Q: Is value iteration guaranteed to find optimal policies?

A: Yes! Guaranteed by the contraction mapping theorem.

Q: Are the intermediate values v_k in value iteration state values?

A: No. They don't satisfy any Bellman equation. They're just intermediate values.

Q: Are the intermediate values v_{π_k} in policy iteration state values?

A: Yes! They are solutions of the Bellman equation for policy π_k .

Frequently Asked Questions (2/3)

Q: Is there an iterative algorithm embedded in policy iteration?

A: Yes! The policy evaluation step uses an iterative algorithm to solve the Bellman equation.

Q: How many iterations should we run in truncated policy iteration?

A: General guideline: run a few (5-20) but not too many.

- A few iterations speed up overall convergence
- Too many iterations give diminishing returns

Q: What is Generalized Policy Iteration (GPI)?

A: Not a specific algorithm, but the general idea of interaction between value and policy updates. Most RL algorithms follow this pattern!

Frequently Asked Questions (3/3)

Q: What's the difference between model-based and model-free RL?

A:

- **Model-based:** Uses data to estimate the system model, then uses the model for learning
- **Model-free:** Does not estimate or use a model during learning

The algorithms in this chapter (value iteration, policy iteration) require the true model, so they're typically called **dynamic programming** rather than RL.

Q: Why are these algorithms important if they require the model?

A:

- Foundation for understanding RL algorithms
- Ideas (especially policy iteration) extend to model-free setting
- Sometimes models are available (games, simulations)

Thank You!

Questions?